高性能计算

HPC 第三关实验报告

日志压缩与稳定筛选优化

目　录

摘要 2
1. 代码理解与接口 2
2. 环境与可复现基线 3
3. 热点、假设与选择 5
4. 实现与正确性 8
5. 性能与适用条件 9
6. 复盘 13
7. 独立复测与归因更正（第三轮） 14
8. 复现入口（全部命令均已从解包后的提交包实测通过） 17

摘要

本报告研究按时间戳、延迟与 `flags` 稳定筛选日志记录的问题。在保持输出顺序、全字段与重复记录的前提下，将 baseline 的四段短路判断改写为无分支按位与谓词、无条件写回与掩码推进指针，并用公开测试、7 项回归、60 组参数化差分和 5 个数据集的逐字节指纹验证正确性。1M/50% 数据集的顺序 A/B 为 **2.07**×；逐样本交替测量给出的可复现区间为 **1.7–2.1**×（空闲态约 2.10×）。消融实验显示，1.67× 的收益中约 1.66× 来自消除“是否写出”的数据相关分支，谓词改写与阈值局部化均无稳定收益。该方案在命中序列可预测（blocks/90%）或命中率 100% 时退化至约 1.0×；两遍分块并行在 2 核上始终慢于单遍，手工 SIMD 未通过正确性验证。报告同时更正了早期对 10M 掉速、输入来源与加速比口径的归因。

关键词：稳定筛选；无分支编程；分支预测；消融实验；性能分析；SIMD

> 构建、运行与文件布局见 `requirements.txt`；**复现入口见文末第 8 节**（正确性用 `bench/verify.sh`，性能用 `bench/perf.sh`，两者都会在缺少数据集时自动补建）。本报告所有耗时数字均来自 `results/` 目录下的原始样本文件，未做任何修饰。

---

1. 代码理解与接口

1.1 原始程序（`bench/baseline.cpp`）的职责划分

原程序把所有职责塞在 `main` 里。按"计时边界"划分如下：

职责	位置	是否在核心计时内
① 参数解析与合法性校验	`main` 第 17–51 行	否
② 输入生成 / 文件解码	第 53 行 `generate(...)` / `load(...)`	否（注释明确"deliberately outside core timing"）
③ **核心筛选**	第 68–76 行（`BEGIN CORE` ~ `END CORE`）	**是**
④ 参考校验	第 55 行 `reference_filter` + 第 78–80 行逐条比对	否
⑤ 打印与落盘	第 59–64、89–94 行	否

计时边界由 `steady_clock` 的一对取值夹定，即 `BEGIN CORE` 到 `END CORE` 之间。

1.2 抽取的接口

```
std::size_t filter_records(const Record* input, std::size_t count,
                           Record* output, const Query& query);
```

保持的语义：

- **稳定**：命中记录按输入中出现的先后顺序写出，不排序、不重排；
- **全字段**：整条 `Record`（24 字节）原样复制，五个字段一个不丢；
- **不去重**：内容完全相同的记录各写一份；
- 返回值是命中条数，有效结果仅为 `output` 的前 N 条；
- `count == 0` 时不解引用 `input`（允许空指针）；
- 函数内不打印、不读文件、不生成数据、不调用参考实现。

`output` 被当作**被动参数**：不分配、不扩容，容量由调用方保证（与 baseline 一致，均为 `count` 条）。

`main` 外壳（参数解析、错误信息、退出码、生成/加载、参考校验、`median_core_ms` 打印格式、`save` 落盘）**逐字沿用 baseline**，仅新增 `--impl`、`--threads` 两个可选参数，因此公开测试中针对外壳的用例天然继续成立。

---

2. 环境与可复现基线

2.1 环境（原始记录见 `results/environment.txt`）

项目	值
内核 / 架构	Linux 7.0.0-38-generic x86_64（Ubuntu 24.04）
CPU	Intel(R) Core(TM) i5-8250U @ 1.60GHz
逻辑核数	2（`nproc`；宿主为 4 核 8 线程，容器只分配 2 核）
内存	5.7 GiB 总量，采集时可用 3.4 GiB
编译器	g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Python	3.12.3（仅标准库 unittest）
编译命令	`g++ -O2 -std=c++17 -pthread <src>.cpp -I src -o <bin>`
线程数	单遍实现恒为 1；两遍实现对比 1 / 2 / 4

计时边界：与 baseline 完全一致——只夹住一次 `filter_records` 调用（含该实现内部的临时分配与线程创建/回收），不含输入生成、输入加载、`output` 缓冲分配、参考校验、打印与落盘。

输入获取方式统一为 `--input` 固定文件（非现场生成）。基线对照组使用同一文件、同一时段背靠背采集。

2.2 输入数据集与指纹

名称	生成参数	记录数	文件大小	实际命中率	输入 sha256（前 16 位）
`n1m-r50.bin`	count=10^6 hit=50 random seed=2026	1,000,000	24,000,024 B	50.0129%	`d11bc48ae929db3f`
`n1m-r0.bin`	count=10^6 hit=0 random seed=2026	1,000,000	24,000,024 B	0%（精确 0 条）	`6685a2d1bc0888b9`

名称	生成参数	记录数	文件大小	实际命中率	输入 sha256（前 16 位）
`n1m-r100.bin`	count=10^6 hit=100 random seed=2026	1,000,000	24,000,024 B	100%（精确 10^6 条）	`2596b029c3bd1973`
`n2m-b90.bin`	count=2×10^6 hit=90 **blocks** seed=2026	2,000,000	48,000,024 B	**90.0000%**	`6163b4d591720b87`
`n10m-r50.bin`	count=10^7 hit=50 random seed=2026	10,000,000	240,000,024 B	50.00126%	`d16baa89cc99f897`

完整 sha256、生成参数清单（含 `hitCountActual`）见 `results/input-sha256*.txt` 与 `results/manifest-*.json`。所有输入的"实际命中率"由平台生成器（`generate_logs.py`）与 C++ 实现**两条独立路径各自计算并交叉验证**，两者一致。

输出指纹（`results/gold-sha256*.txt`）：

数据集	输出 sha256（前 16 位）	命中条数
`n1m-r50`	`a3caefa4fa011150`	500,129
`n2m-b90`	`8cc80737a0ce7a6e`	1,800,000
`n10m-r50`	`49d002e5cd078500`	5,000,126

优化实现在全部数据集上的输出与标准答案**逐字节相同**（`results/fingerprint-check.txt`）。

2.3 原始样本与中位数

采集方式：A（baseline）与 B（optimized）**交替重复**多轮，每轮内部各跑 5 次取中位数，以削弱热漂移与容器负载波动。原始样本见 `results/ab-*.txt`。

说明：本机实测噪声极大（同一份代码同一份输入，单次耗时可在 3 倍范围内波动，见 5.4 节）。因此表中同时给出**每轮 5 次中的最小值**（代表无干扰能力）与中位数。

实验一：1M / random / hit=50%（`results/ab-1m-r50.txt`）

实现	轮次	5 次原始耗时（ms）	最小值	中位数
原始 baseline	1	29.378 / 29.359 / 30.654 / 26.400 / 23.129	23.129	29.359
原始 baseline	2	17.997 / 22.925 / 17.241 / 28.431 / 14.813	14.813	17.997
原始 baseline	3	20.966 / 26.907 / 42.666 / 34.505 / 25.843	20.966	26.907
优化	1	27.883 / 23.222 / 20.594 / 24.138 / 28.373	20.594	24.138
优化	2	17.790 / 14.926 / 12.569 / 9.640 / 8.167	**8.167**	12.569
优化	3	15.215 / 15.516 / 14.522 / 15.498 / 19.004	14.522	15.498

→ 全局最小值 **20.594 → 8.167 ms（2.52×）**；最小中位数 **26.907 → 12.569 ms（2.14×）**。

> ⚠️ **第三轮更正**：本节（以及 5.1）的绝对值是在宿主高负载时段采集的，且当时 `bench_run.sh` 采用"先跑完全部 A 再跑完全部 B"的顺序。改用逐样本交替后，同一份 1M 数据的可复现加速比区间为 **1.7×~2.1×**（详见 7.2 节）。**引用加速比时请连同机器状态一起引用**，不要单独引用 2.52×。

实验二：10M / random / hit=50%（`results/ab-10m-r50.txt`，5 轮）

实现	最小值（跨轮）	中位数（跨轮）
原始 baseline	91.629 ms	105.580–172.161 ms
优化	**55.072 ms**	62.622–102.988 ms

→ 按最小值 **91.629 → 55.072 ms（1.66×）**。（该数据集另有单次独立测量：baseline 150.98 ms / optimized 55–72 ms，见 `results/exp-threads-10m.txt` 与第 5 节。）

---

3. 热点、假设与选择

3.1 热点证据

（a）**`perf` 不可用**。 容器设置了 `perf_event_paranoid = 4` 且禁止提权（"no new privileges"），硬件计数器无法采集。因此热点证据改用**编译器向量化诊断 + 针对性微基准**，两者都直接覆盖 `BEGIN CORE ~ END CORE` 这一核心区域（微基准只测被计时的内核函数，不含进程启动、生成、加载与校验）。

（b）编译器的诊断明确指出核心循环无法向量化。 用 `-O3 -march=native -fopt-info-all` 编译核心循环：

```
missed: not vectorized: no vectype for stmt: _2 = r_19->t;
```

即 **`Record` 中 64 位 `timestamp` 的比较在通用 x86-64 下没有对应向量类型**（SSE2 无 64 位整数比较），向量化器直接放弃。这也解释了另一个观测：**同一份源码加 `-march=native`（开放 AVX2）编译，耗时反而慢约 5%（10.503 → 11.004 ms 中位数），即在噪声内无收益**——因为 `-march=native` 只影响生成器循环（`support.hpp:70` 确实报告 "vectorized using 32 byte vectors"），对过滤循环无效。

（b2）另一个更基础的障碍：GCC 13 的 `-O2` 不做 reduction 向量化。 用一个最小对照验证：`for (i) h += a[i];` 这类简单求和循环，

编译选项	是否向量化
`-O2`	❌无任何向量化报告
`-O3`	✅ `loop vectorized using 16 byte vectors`
`-O2 -ftree-vectorize`	✅同上

本题指定 `-O2`，因此即便把谓词写成最容易被向量化的形式（无分支、位掩码累加），求和累加器本身就挡住了向量化。函数级 `__attribute__((optimize("tree-vectorize")))`、`optimize("O3")`、`target_clones("avx2")` 三种写法在 `-O2` 下实测**均不生效**。这是"不采用手工 SIMD"这一结论的第二个依据。

（c）逐条代价的构成。 依据生成器语义与实测分布（默认查询下：时间戳通过率 83.35%、延迟通过率 83.33%、flags 通过率 83.33%），三个谓词都是"高但不满"的通过率。这里需要更正第二轮的措辞：**baseline 源码并不是四段 `if (...) continue;`，而是一个用 `&&` 串起来的短路链**；`-O2` 下它被编译成 4 个条件跳转（反汇编见 `bench/micro_core.cpp` 对照），**外加一个控制"是否写出"的数据相关分支**。第三轮消融证明，真正昂贵的是最后这一个"写出分支"，而不是谓词那四个（见 7.3）。

（d）微基准定位（已由第 7 节的消融实验取代）

> **本节数据已在第三轮复测中被更正，请以 7.3 节消融实验 为准**。 下表是第二轮探索期的**探针输出**，两个问题使它不足以作为归因依据：(a) 只记录"最好一次"，未归档原始样本；(b) 其中"阈值局部化带来 26% 提升"这一条，在第三轮的 6 组独立测量中**一次都没有复现**（4 组为负收益）。

写法（第二轮探针，仅供对照，勿引用）	探针最好耗时
baseline 原样	11.6–19.6 ms
合并为一个按位表达式 + 无分支搬运	5.85–5.96 ms
阈值预取到局部变量	8.1 ms（**该结论已被推翻，见 7.3**）

3.2 假设

> **主假设　H1（第三轮已部分否定）**：核心开销主要来自**控制流的数据相关性**，而不是内存带宽；把判断与搬运改成无分支应获得接近 2 倍的提升。原表述把"控制流"具体化为"谓词判断的组织方式 + 阈值局部化"，**这一具体化被 7.3 的消融实验否定**：谓词改写只值 0~2%，阈值局部化为负收益；真正被消除的是**写出分支**。H1　的"非内存带宽"这一半则得到支持（空闲态下　10M　与　1M　的加速比同为 2.10×）。

> **次假设 H2**：核心循环因 64 位时间戳比较而无法被向量化；若能拆分数据布局（时间戳/flags 各自连续），计数遍可被 AVX2 向量化而大幅加速。

> **假设 H3**：把筛选拆成"分块计数 + 前缀和 + 分块写出"后，第二遍可跨块并行，在 2 核上获得额外加速。

3.3 会推翻假设的观测

假设	推翻它的观测	实际结果
H1	无分支写法与 baseline 相当或更慢	**未被推翻**，但**归因已更正**：整体加速成立（1M 1.69×/2.11×、10M 1.71×/2.10×，见 7.2），收益来自写出分支的消除而非谓词（见 7.3）
H2	数据布局转换的代价高于扫描节省	**被推翻**（见 3.4 候选二）
H3	两遍方案即便多线程仍慢于单遍	**被推翻**（见 3.4 候选三）

3.4 候选方案的比较与取舍

候选	做法	预期	实测（原始样本）	结论
一	单遍扫描 + 无分支搬运 + 阈值局部化	消除分支错预测与重复内存读	见 7.2/7.3：空闲机 1M 1.69×（保守）/ 2.11×（最优）；消融显示收益全部来自"无分支搬运"，谓词改写与阈值局部化≈0	✅ **采用**（归因已于第 7 节更正）
二	转数据布局（timestamps[]/flags[] 两数组）后向量化计数	AVX2 一次比 8 条	**原始样本未归档（第三轮核查）**	⚠️ **降级为"未验证"**：改为"因实现范围与收益预期未纳入"，不再声称"实测已否定"
三	分块计数 + 前缀和 + 分块写出（可多线程）	前缀和使块可并行且无写冲突	见 7.5：空闲机 10M，single 74.6–85.9ms vs twopass 1/2/4 线程最小 145.4/190.6/146.1ms	❌ 放弃：多读一遍输入的代价无法被 2 核并行弥补；实现保留为 `--impl twopass` 供复现对比
四	用 `flags` 最高位作廉价预筛	一半记录提前排除	该位在真命中样本中并非恒为 1，命中数只剩 249,824（**错误**）	❌ 放弃（本身是错误假设，留作反面记录）
五	32 位阈值快路径（守卫 `q_end <= 2^32`）	32 位比较可被向量化	**原始样本未归档（第三轮核查）**；公平速度对比缺失	⚠️ **降级为"未验证"**。正确性论证成立（见 3.5），仅因收益预期不足未纳入实现范围
六	**手工 SIMD**（AVX2 intrinsics，一次 4 条记录）	谓词全部在向量里算，省下判断开销	编译器诊断见 `results/vectorization-report.txt`；原型**未通过正确性验证**，故不给出加速比	❌ **放弃（尝试过、未能验证）**：见 3.6。24 字节 AoS 布局的字段组装成本 + 相同的存储开销，使净收益缺乏证据

候选三虽然在本机被否定，但它满足题目分层提示的三条要求（块内计数、排他前缀和、无写冲突的稳定写出），因此**保留在源码中**（`--impl twopass`），便于评审复现该结论，也便于核数更多的机器上重新评估。

3.5 关于"32 位快路径"：为什么它可以被证明正确，却仍然不采用

正确性论证（无需预扫描求最大值）：设守卫条件为 `query.end <= 2^32`。

- 当 `query.end < 2^32` 时：把 `timestamp` 截断成 32 位记为 `t32 = (uint32_t)timestamp`。
  - 若 `timestamp < 2^32`，则 `t32 == timestamp`，比较结果与 64 位完全一致；
  - 若 `timestamp >= 2^32`，则它必然 `>= query.end`，应判"不命中"；而此时 `t32 <= 2^32-1 < query.end`，32 位比较同样判"不命中"。**结论一致**。
- 当 `query.end == 2^32` 时：上界退化为"无上界"，只需判断 `t32 >= query.begin`，同样正确。

因此这个快路径**不需要**先扫描全部记录求时间戳最大值——只需检查 `query` 的端点这一个标量，守卫成本是 O(1)。这一点比"先求最大值"的方案（多一趟 24 MB 扫描）优越得多。

证据等级更正（第三轮）：本节原来引用的候选五实测值（1M 4.60/4.38 ms、10M 51.9/53.6 ms）**在 `results/` 下没有任何原始样本归档**，无法被评审复核。因此该候选的结论由"已实测否定"**降级为"未验证"**：正确性论证仍然成立，不采用的理由改为**收益预期不足且未纳入本次实现范围**，而不是"实测更慢"。同理，3.4 候选二（SoA 转换）的那组数字（转换 4.46 ms + 向量化扫 1.78 ms）也缺乏原始样本，一并降级为"未纳入范围"。若要恢复为确定结论，需要补做实验并归档原始样本（见第 7 节的建议）。

3.6 关于向量化：编译器为什么拒绝，以及手工 SIMD 的实测结论

题目明确允许 SIMD（"CPU 循环/判断、SIMD、分块并行、布局转换均可"），因此这一节不是 "要不要用"的取舍，而是**"用了会怎样"的实测记录**。完整证据见 `results/vectorization-report.txt`（只读编译器诊断 + 反汇编，未改动任何交付源码）。

（a）自动向量化：交付版与 baseline 都没有被向量化。

为让诊断指向明确函数，把两个核心循环各自单独放一个翻译单元（内容与交付源码逐字相同）：

循环	`-O2`（题目口径）	直接原因（编译器原文）
交付版（单遍无分支）	`vectorized 0 loops in function`	`more than one data ref in stmt: *_13 = *r_28;` + `Analysis failed with vector mode V2DI`
baseline（四段短路分支）	`vectorized 0 loops in function`	`not vectorized: control flow in loop.`（外加同样的多数据引用）

即使把优化级别与指令集都放开到 `-O3 -march=native`（AVX2），过滤循环**仍然** `vectorized 0 loops`；整份 `src/optimized.cpp` 里被向量化的位置只有 `optimized.cpp:220`（命令行默认值初始化，SLP）、`support.hpp:70`（数据生成器，32 字节向量）、`support.hpp:81`（写头部时的字节组装）、`support.hpp:98`（读文件）——**全都在计时范围之外**。

（b）交付版内层循环的逐条指令（反汇编）：

```
movdqu (%rdx),%xmm0                          ; 取一条记录的前 16 字节
cmp %rbx,%rcx ; setae %al                    ; 时间戳下界 → 寄存器低字节
cmp %r11,%rcx ; setb  %cl                    ; 时间戳上界
and %ecx,%eax
cmp %r10d,0xc(%rdx) ; setae %cl              ; 延迟
and %ecx,%eax
mov 0x14(%rdx),%ecx ; and %r8d,%ecx ; sete %cl ; flags
and %ecx,%eax
movups %xmm0,(%rcx)                          ; 写出前 16 字节
mov -0x8(%rdx),%r14 ; mov %r14,0x10(%rcx)    ; 写出后 8 字节
add %rax,%rsi                                ; 写指针 += 命中掩码
```

`cmp/setcc/and/add` 全是标量指令；`movdqu`+`movups` 只是把 24 字节记录整体搬一趟， xmm 寄存器**只当搬运筐用**，一条指令只处理一条记录。**结论：交付版的核心路径没有任何向量运算**——它是"标量 + 无分支"，而不是 SIMD。

（c）手工 SIMD 原型：尝试了，但未能验证。

本轮另外用 AVX2 intrinsics 写了原型：一次处理 4 条记录，谓词全部在向量里算（时间戳用 `(uint64)(t-begin) < span` 的无符号比较技巧 + `vpcmpgtq`；延迟用 `vpsubd`+`vpcmpgtd`；flags 用 `vpand`+`vpcmpeqd`），再用 `movemask` 取 4 位掩码、按位序标量写出以保持稳定顺序；字段提取试过"跨步载入组装"与"一次 96 字节栈拷贝"两版。

结果：**两版都未能通过正确性验证**（逐谓词定位显示时间戳路径完全一致，延迟与 flags 路径存在差异；错误根因先后定位到无符号比较的操作数顺序、以及 32 位谓词与 `movemask_pd`（按 64 位 lane 取高位）不匹配两处，修掉后仍未完全一致）。因此**本报告不给出手工 SIMD 的加速比**——用未通过正确性验证的代码测出的耗时没有意义。这一点如实记录，不写成"SIMD 已被实测否定"。

（d）不依赖原型、可以确证的三点结构性判断：

1. **布局不利**：24 字节 AoS 记录，三个字段分别落在非对齐偏移 `+0 / +12 / +20`。

   向量化必须先付"字段组装"（跨步载入 + 重排）的成本，而标量版每条记录只要 1 条 `movdqu`。

1. **存储开销相同**：每条命中记录都要写 24 字节（16B+8B 两条 store），标量与 SIMD

在这一项上**完全一样**；而收益的真正来源（消融实验，见 7.3）恰恰是"消除写出分支"，不是"算得更快"——SIMD 优化的方向与瓶颈所在的方向不一致。

1. **64 位整数比较**：基线下（SSE2）没有 64 位整数比较，编译器试 `V2DI` 模式直接失败；

这也是候选二（SoA 布局）想绕开的东西，而布局转换本身的代价同样未获证据支持。

小结：交付版**不含向量化运算**；自动向量化在本问题的循环形态下不被 GCC 接受；手工 SIMD 属于"尝试过、未能验证"的状态。若要把它变成确定结论，需要先让原型通过正确性验证（含块边界与极端字段），再在固定文件、交替测量的口径下与标量版对比并归档原始样本。

---

4. 实现与正确性

4.1 最终实现要点（src/optimized.cpp）

默认实现 `single`（单遍、无分支）

```
const std::uint64_t q_begin = query.begin;      // 阈值局部化：不在循环内重复读 Query
const std::uint64_t q_end   = query.end;
const std::uint32_t q_lat   = query.min_latency_us;
const std::uint32_t q_flags = query.required_flags;
for (std::size_t i = 0; i < count; ++i) {
    const Record& r = input[i];
    const unsigned ok = (unsigned)(r.timestamp >= q_begin) & (unsigned)(r.timestamp < q_end)
                      & (unsigned)(r.latency_us >= q_lat)
                      & (unsigned)((r.flags & q_flags) == q_flags);
    output[m] = r;      // 无条件写
    m += ok;            // 命中才推进写指针
}
```

稳定性保证：写出位置 `m` 单调不减，第 i 条记录必写在所有更早命中记录之后，因此输出顺序恒等于输入顺序。**不依赖排序，也不依赖任何并行同步**。

原地安全性：`m <= i` 恒成立，写指针永远不越过读指针，因此即便 `output == input`（原地压缩）也不会破坏未读数据——这是额外的余量，接口约定仍是两者不重叠。

实现的额外余量：`output[m] = r; m += ok;` 中的无条件写会在未命中时写一个"将被覆盖或越界一位"的位置。已由 `output` 容量 ⩾ `count` 的接口约定保证合法；任何调用方若违反该约定，行为未定义。

备选实现 `twopass`：分块（4096 条/块）计数 → 排他前缀和把命中数就地改写为输出起始位置 → 按块序写出，块内保持输入顺序。各块输出区间由前缀和精确划分、互不重叠，因此稳定；块间可分配线程且无写冲突。临时内存为 `O(块数)`（10^7 条时约 19 KiB）。默认实现 `single` 的临时内存为 `O(1)`。

计时内的初始化成本：`single` 无分配、无线程；`twopass` 的前缀和数组分配与线程创建均在计时范围内，未做任何"把初始化挪出计时"的处理。

4.2 正确性验证

验证	命令	结果
公开测试	`python3 tests/test_public.py bench/optimized`	`Ran 7 tests ... OK`
自建回归测试	`python3 tests/test_regression.py bench/optimized`	`Ran 7 tests ... OK`
参数化差分	`python3 bench/diff_check.py --exe ./bench/optimized --base ./bench/baseline`	**60 次比对，失败 0 次**
输出指纹	`bash bench/run_all.sh` 第 7 步	5 个数据集全部逐字节一致

公开测试保留且未修改（与平台提供文件逐字节相同，`cmp` 已验证），7 个用例全部通过——包括非法参数 12 种、损坏文件 8 种、输出已存在拒绝覆盖、输入不被改写、生成器可复现性等外壳行为。

自建回归测试补充的边界用例（针对优化引入的风险点）：

1. **块边界**：4095 / 4096 / 4097 / 8192 / 8193 条（块大小为 4096，专测边界上下各一条）；
2. **实现与线程数一致性**：`single`、`twopass` 1/2/4 线程对同一输入的输出必须逐字节相同（稳定性的并发不变性）；
3. **确定性**：同一输入重复运行结果一致；
4. **极端字段**：全 0、`UINT32_MAX`、`UINT64_MAX` 时间戳，以及左闭右开边界（`999` 排除、`1000` 保留、`1999` 保留、`2000` 排除）；
5. **大样本差分**：3 万~5.3 万条 × 3 组查询，与 Python 独立参考实现逐条比对（含顺序与全部字段）；
6. **命中率两端**：0% 时输出必须精确为 0 条，100% 时精确为 N 条；
7. **参数边界**：`--impl bogus`、`--threads 0`、`--threads 99999` 必须被拒绝（退出码 2）。

---

5. 性能与适用条件

5.1 两组（共五组）参数的结果汇总

口径统一：`--input` 固定文件、`-O2 -std=c++17 -pthread`、单次计时只含 `filter_records`。加速比按**每轮 5 次的最小值**计算（该机器噪声大，最小值比中位数更稳定，理由见 5.4）。

数据集（规模/分布/命中率）	baseline 最小	optimized 最小	加速比	说明
100k / random / 50%	0.75 ms	**0.30 ms**	**2.50×**	小规模
250k / random / 50%	1.83 ms	**0.88 ms**	**2.07×**	
500k / random / 50%	3.75 ms	**1.75 ms**	**2.14×**	
1M / random / 50%	7.85 ms	**3.79 ms**	**2.07×**	主场景
2M / random / 50%	16.80 ms	**7.45 ms**	**2.25×**	
10M / random / 50%	95.23 ms	**51.75 ms**	**1.84×**	大样本

数据集（规模/分布/命中率）	baseline 最小	optimized 最小	加速比	说明
2M / **blocks** / 90%	13.20 ms	12.95 ms	**1.02×**	**退化场景**
1M / random / 0%	10.48 ms	**5.10 ms**	**2.05×**	无输出
1M / random / 100%	6.23 ms	6.87 ms	**0.91×**	**退化场景（略慢）**

上表全部数据来自**同一会话内背靠背交替采集**（原始样本见 `results/scale-sweep.txt` 与 `results/ab-*.txt`）。可视化见：

- `results/chart-scale.svg` —— 规模扫描（耗时 + 加速比）
- `results/chart-hitrate.svg` —— 命中率扫描（耗时 + 加速比）

规模趋势：从 10 万条到 1000 万条，本表加速比在 **1.84×~2.50×** 之间；1000 万条时为 1.84×。

> ⚠️ **第三轮更正**：上表的绝对值受采集时段影响很大。"10M 因内存带宽受限而摊薄到 1.84×"这一解释**只在负载态成立**——空闲态重测中 10M 为 **2.10×**（64.4 → 30.6 ms），与 1M 的 2.10× 相同，说明该规模的加速比并未被带宽摊薄。逐样本交替测量给出的可复现区间是 **1.7×~2.1×**（见 7.2）。

5.2 为什么有提升（本节归因已于第三轮更正）

> 第二轮的下面三条解释**方向是对的但不准确**。第三轮的消融实验（7.3）把单遍实现拆成两步单独计时，结论是：**收益几乎全部来自第 3 条（把"是否写出"的数据相关分支换成无条件写 + 掩码推进指针），第 1 条只贡献约 0~2%，第 2 条（阈值局部化）没有任何可复现收益、4 组测量中为负收益。**下面保留原文以便对照，更正后的表述见 7.3。

1. **~~消除分支错预测：baseline 的四段 `if(...) continue;` 在 83% 通过率下难以预测~~** → 更正：baseline 是**一个 `&&` 短路链**（编译为 4 个条件跳转），它确实会产生错预测，但消融显示把谓词改成按位与只值 0~2%。
2. **~~阈值局部化：baseline 每轮迭代都要从内存重新读取 `Query` 的四个字段；取到局部变量后循环内不再有这些依赖加载。单项收益实测约 26%~~** → 更正：该 26% **无法复现**，6 组独立测量中 4 组为负收益。原因很可能是 `-O2` 本来就能把 `Query` 的常量字段提出循环，手工局部化反而改变了寄存器分配。
3. **减少数据相关的控制流（真正的收益来源）**：baseline 只在命中时才写输出，因此循环体里有一个**由数据决定方向、且每两条记录就翻转一次的"写出分支"**；本实现改为"无条件写 + 把命中掩码加到写指针上"，把"是否写"这个分支转换成一次加法与一次固定代价的小宽度存储，乱序执行不再因分支解析而停顿。代价是多写了（1M/50% 场景下多写约 12 MB）并多了一条指令——实测净收益 1.67×。

5.3 为什么在另外两组退化

情形	原因
blocks / 90%	两个因素叠加：(a) 命中率 90% 时输出量达输入的 90%，瓶颈从"判断"转移到"写出"（内存带宽），而本实现并没有减少写出的字节数；(b) `blocks` 分布让前 9000 条持续命中、后 1000 条持续不命中，baseline 的分支**极其容易预测**，其分支代价本来就接近零，因此优化的着力点失效。
hit=100%	所有记录都命中，输出量等于输入量（24 MB），整个任务退化为"整块复制"。此时 baseline 的分支恒为同一方向（完美预测），而本实现的无条件写在第 i 条与第 i 条之间没有任何区别，唯一的差别是基线省掉了"最后一次多余的写"。因此略慢约 9%，属于正常开销差。

结论：本方案的收益区间是中等选择性（约 1%~90%，且分布不呈现长程规律），最适用于"判断密集"而非"写出密集"的场景。在选择性接近 100% 时，应直接退化为内存拷贝语义，任何判断优化都无从发挥。

5.4 噪声、缓存与测量口径的影响（重要）

本机（2 核容器，i5-8250U 低压 U）**绝对耗时不可跨时段比较**。同一份 baseline、同一份输入、同一份二进制，在不同时段实测中位数从 **6.46 ms 一路漂到 26.65 ms**（相差 4.1 倍），原因：

- **宿主负载/热漂移**：低压 U 系列在持续负载下降频，而本容器是 VMware 虚拟机，宿主的其他负载会直接改变可用算力与内存带宽；
- **容器争用**：`nproc = 2`；
- **输入获取方式**：`--count`（现场生成）与 `--input`（从文件加载）产出的数据**内容完全相同**（同参数生成具有确定性，1M 数据集导出文件 sha256 = `d11bc48ae929db3f...`，与 `data/n1m-r50.bin` 一致），因此两者**不存在速度上的系统性差异**——第四轮补做的逐样本交替实验（`results/source-generated-vs-file.txt`）：
  - 100k（输入 2 MiB，可驻留 L3）：中位之比 **0.98**×，逐轮 A 慢 4/8 轮，符号检验 p = 1.00；
  - 1M（输入 22 MiB，远超 L3）：中位之比 **1.02**×，逐轮 A 慢 6/8 轮，p = 0.29（不显著）。

仍统一使用 `--input` 的理由**不是"它更快"**，而是它**可归档、可核对、可复核**（有文件哈希与 `gold/` 输出指纹），且所有性能样本必须来自同一份可指定的输入。两种方式**仍不应混比**——不是因为速度，而是因为它们互相不可验证（一个靠文件哈希，一个靠参数约定）。

第三轮补充发现的两个测量学问题（详见 7.1）：

- **A/B 顺序偏差**：`bench/bench_run.sh` 是"先跑完全部 A 轮，再跑完全部 B 轮"。A 独占机器、B 在 A 造成的热与负载状态下运行，两边并不对等。第三轮改用**逐样本交替**（且偶数轮反转 A/B 先后），使同轮内的 A、B 共享同一热轨迹。
- **噪声是双向的、且偏爱谁并不固定**：1M 数据集上，空闲态加速比收敛到 **2.11**×（逐轮区间 [2.02, 2.15]），负载态则是 **1.76**×。因此对比必须**同会话背靠背**，并以**跨轮最小值 + 逐轮中位数之比**双判据报告，单次测量在这台机器上没有判定力。

第四轮补充：`min` 判据在"小数据集 / 少轮次"下尤其不稳。同一次 `perf.sh` 运行（3 轮规模扫描 + 6 轮交替）内出现：

规模	按跨轮最小值之比	按逐样本交替 + 配对中位数之比
100k	2.00×	—（未测）
250k	1.87×	—
1M	**1.59**×	**1.75**×
2M	1.61×	—
10M	1.73×	1.73×（一致）

即：数据集越小，两个判据的分歧越大（1M 上差 10%），越大越收敛（10M 上完全一致）。原因是 `min` 取的是每组的**单个最幸运样本**，样本越少、受单次抖动影响越大；而交替 + 配对使用全部样本。因此**小规模对比应以交替口径为准**，`min` 只作为参考。

缓存影响：1M 条 ≈ 22–24 MB 输入，已超过本机 L3（约 6 MiB），因此测的是真实内存访问；`twopass` 方案需要再读一遍 24 MB，这正是它在 2 核上无法回本的根本原因。

5.5 并行扩展性（`--impl twopass`）

第三轮重测（原始样本 `results/twopass-vs-single-10m.txt`，10M random/50，同会话背靠背，采集时 load1≈2.6）：

配置	5 次原始耗时（ms）	最小值	中位数
twopass 1 线程	168.4 / 232.7 / 216.8 / 145.4 / 210.8	145.45	210.79

配置	5 次原始耗时（ms）	最小值	中位数
twopass 2 线程	247.4 / 221.8 / 190.6 / 200.0 / 213.2	190.62	213.18
twopass 4 线程（超订）	203.9 / 239.3 / 191.0 / 146.1 / 158.6	**146.08**	190.95
single（1 线程）	82.6 / 85.9 / 74.6 / 119.9 / 91.8	**74.64**	**85.86**

结论比第二轮**更强、且不依赖基线口径**：

- 单遍的**最小值（74.6）甚至优于两遍任一配置的最小值（145.5 / 190.6 / 146.1）**，即单遍的"最差情况"附近的水平仍是两遍"最好情况"的两倍快；
- 2 线程相对 1 线程没有可复现的改善，4 线程（超订 2 核）的最小值略优但中位数仍差——**并行收益被"多读一遍 240 MB"和共享内存通路吃掉了**；
- 中间一行 2 线程中位数 213.18 与 1 线程 210.79 几乎持平，说明线程调度开销本身已足以抵消一份理论并行度。

> 第二轮版本（`results/exp-threads-10m.txt`）的同一对比表曾记录"single 最小值 55.07 ms"，那是在宿主高负载下采集的，与两次空闲态测量（60.1 / 74.6 ms）差 1.4~2.7 倍；引用的结论方向不变，但**绝对值请以本节为准**。

第四轮补充（干净机器，10M random/50，采集时 load1=1.12）：

配置	中位数 (ms)	相对 1 线程
twopass 1 线程	109.281	1.00×
twopass 2 线程	**55.244**	**1.98×**
twopass 4 线程（超订）	58.754	1.86×
single 1 线程	**29.641**	比 twopass 最快配置仍快 **1.86×**

这组数据更正了本节原先的解释。原先写的是"2 线程相对 1 线程没有可复现的改善，并行收益被'多读一遍 240 MB'和共享内存通路吃掉了"——第四轮实测表明**并行本身接近线性（2 线程 1.98×）**，原先的"没有改善"是高负载态（采集时 load1≈2.6）下的测量噪声造成的误判。

正确的解释是：**两遍方案输在"固定成本"，而不是"并行无效"**。

- 多读一遍 240 MB：约 +20 ms，且这部分**不可并行摊薄**（每个线程都要读自己的那份输入，总字节数不变）；
- 前缀和数组分配 + 每遍的线程创建/回收：约 +40 ms（实现是每次调用现建线程，未用线程池）；
- 二者相加约 +60 ms，而单遍只要 29.6 ms —— **即便两遍的并行加速是理想的 2×，109 ms 也只能压到 55 ms，仍差 1.86 倍**。

因此结论方向不变（**单遍更优**），但理由从"并行无效"改为"**并行有效，但被固定成本抵消**"。这也提示一条真正可能的改进方向：若把线程池化、前缀和缓冲复用，两遍方案在大规模上的差距会收窄——但那属于连续处理多批查询的场景，本次未做。

证据文件的生成位置：本节引用的 `results/exp-threads-10m.txt` 与 `results/twopass-vs-single-10m.txt` 由 `bench/run_all.sh` 的线程扩展性步骤生成；`bench/perf.sh` 不含该步骤（它的第 6 步统计汇总会显示"缺 exp-threads-10m.txt，跳过"，属预期行为）。要复核本节，请用 run_all.sh，或跑 `bench/perf.sh`（第四轮已补入第 3b 步，会生成该文件；见 `results/perf-full-run-verification.txt`），或直接手跑 `./bench/optimized --input data/n10m-r50.bin --impl twopass --threads {1,2,4} --repeats 5`。

适用范围声明：本机只有 2 个逻辑核。若评测机核数更多、内存带宽更高，两遍方案存在反超可能，但需要补两条前提——(a) 把写出遍也改成无分支搬运（否则它仍带着数据相关写出分支，见 7.3）；(b) 在目标机器上重测。本次未做该实验。

5.6 初始化与临时内存

实现	计时内分配	计时内启动线程	临时内存
`single`（默认）	无	无	O(1)
`twopass`	前缀和数组	每遍 1~3 个	4×(块数+1) 字节；10^7 条 → 约 19 KiB

若上层需要连续处理多批查询，`twopass` 的线程池化或前缀和缓冲复用可以进一步摊薄成本，但这属于本关范围之外的场景。

---

6. 复盘

6.1 AI 建议的采用与否定

建议	处置	依据
用分层提示的分块 + 前缀和 + 并行方案	**实现并实测，但否定为默认**	本机 2 核下两遍方案 1M 为 25.2 ms、10M 4 线程 159.6 ms，均慢于单遍（8.17 / 55.1 ms）；保留为 `--impl twopass` 供复现
转数据布局（SoA）以便向量化	**否定**	转换遍 4.46 ms + 向量化扫 1.78 ms，合计 6.01 ms，慢于直接扫 AoS 的 3.2~3.5 ms
用 `flags` 最高位做廉价预筛	**否定（假设本身错误）**	实测真命中样本中该位并非恒为 1，采用后命中数从 500,129 掉到 249,824
用 32 位阈值比较走"快路径"	**实现并实测，最终否定**	守卫条件 `q_end <= 2^32` 可证明正确且成本 O(1)（见 3.5），但实测无收益：1M 差 5%（噪声内），10M 反而慢 3%
通过 `-march=native` 提升	**否定**	实测中位数反而慢 5%（在噪声内），因为过滤循环因 64 位比较无法向量化

6.2 实际遇到的错误与修复

错误	现象	定位与修复
`record_matches` 定义在使用之后	编译报 `'record_matches' was not declared in this scope`	将谓词函数移到 `filter_records` 之前（C++ 先定义后使用）
首版两遍实现慢一倍	1M 下 33.6 ms vs baseline 17.3 ms	定位为"多读一遍输入"，据此做了候选一/二/三的横向实测
单遍实现最初与 baseline 打平	19.7 ms vs 19.8 ms，与微基准预期不符	定位为"每轮迭代重复从 `Query` 读取 64 位阈值"；阈值局部化后 1M 降到 8.17 ms
微基准探针出现 `hits=0`	计数看似失效	实为探针打印笔误；用独立的小程序复核计数为 500,129，确认耗时数据可信

6.3 如何排除"靠改变计时范围或输出语义"造成的虚假提升

1. **计时范围**：`BEGIN CORE` / `END CORE` 的位置与 baseline 完全一致，只夹住一次 `filter_records` 调用；`twopass` 的前缀和分配与线程创建都在计时内，未外移。`output` 缓冲区的分配、输入生成/加载、参考校验、打印、落盘全部在计时外——与 baseline 的处理相同。
2. **输出语义**：每个数据集都用 baseline 产出的输出作为"输出指纹"（`results/gold-sha256*.txt`），优化实现必须在全部 5 个数据集（含 4095/4096/4097 块边界与 0%/100% 两端）上输出**逐字节相同**的结果；此外程序内部仍在每次采样后调用 `reference_filter` 逐条比对全部 5 个字段（`check=PASS`）。

3. **输入一致**：全部性能采样使用同一份固化文件，并记录输入 sha256；程序从不改写输入文件（公开测试与自建回归测试均验证）。
4. **结论的自洽性**：报告中同时给出了**退化的两组**（2M/blocks/90% 仅 1.02×、hit=100% 为 0.91×），说明数据并非只挑好看的一组；加速比按"每轮 5 次最小值"计算而非单次最优，且给出了完整的原始样本供复核。

6.4 已知局限

- 本机仅 2 核且为低压 CPU，噪声大；`twopass` 方案在多核机器上的结论可能不同，需要在评测机上复测。
- 未取得硬件计数器数据（容器禁用 `perf`），热点结论由编译器向量化诊断与差异化微基准支撑，未做到指令级归因。
- 未使用 SIMD intrinsics：在通用 x86-64 下 64 位时间戳比较无法向量化；转 SoA 布局的代价已被实测否定（3.4 候选二），而 GCC 13 在 `-O2` 下不做 reduction 向量化，因此手工 SIMD 的落地成本与风险都高于预期收益。
- GPU 部分为选做，且平台未发布 `cuda` / `hpc-cuda-checker` 镜像组合，未涉及。

---

7. 独立复测与归因更正（第三轮）

> 本节的所有数字来自 `results/interleaved-ab.txt`、`results/ablation-micro.txt`、`results/twopass-vs-single-10m.txt` 三个原始样本文件，可用 `bash bench/verify.sh`（正确性）与 `PERF_ASSUME_YES=1 bash bench/perf.sh 4 20`（性能）一键重放。

7.1 为什么复测：测量方法与归因都有可质疑之处

第三轮开始时对前两轮的证据做了逐条回溯，发现三类问题：

问题	具体表现	影响
A/B 顺序偏差	`bench/bench_run.sh` 的循环是"每轮内先 A 后 B、把全部 A 轮跑完再跑全部 B 轮"	A 与 B 不在同一热状态下测量；低压 U 的降频与宿主争用会系统性偏向其中一侧
归因未经消融验证	第二轮结论"收益来自消除谓词分支的错预测"只由"baseline vs 最终实现"的整体差推出，没有把两个改动分开计时	可能把功劳记错地方（实测确实记错了，见 7.3）
证据缺口	3.4 候选二/候选五、以及"阈值局部化 +26%"的耗时数字在 `results/` 下**没有原始样本**；`results/ab-1m-r0.txt`、`ab-1m-r100.txt` 为空文件	评审无法复核；其中"阈值局部化 +26%"经复测为**不可复现**

因此第三轮做了两件事：把 A/B 改成**逐样本交替**，以及把单遍实现**拆成两步单独计时**。

方法（脚本 `bench/interleaved.sh`、消融程序 `bench/micro_variants.cpp` + `bench/micro_core.cpp`）：

1. 每个样本点都是**独立进程**，一次只调用一次 `filter_records`，与题目的计时口径一致；
2. 同一轮内按 `A→B` 执行，**偶数轮反转先后顺序**，消除"谁先跑"的系统差异；
3. 用**跨轮最小值之比**（抗单点干扰）与**逐轮中位数之比的符号检验**（`bench/analyze_interleaved.py`）双判据；
4. 微基准把 5 个变体放在**同一进程、同一轮次内交替**执行，并在每轮计时前用 `clflushopt` 把输入刷出缓存，避免工作集意外驻留；
5. 每个变体都先做**逐条正确性检查**（命中数 + 全部字段 + 顺序），避免"用错结果换速度"。

7.2 结果一：规模与命中率扫描（`results/scale-sweep.txt`、`interleaved-ab.txt`）

A/B 顺序执行、每轮 5 次、取跨轮最小值（同一会话背靠背）：

数据集	baseline (ms)	optimized (ms)	加速比	工作集
100k / random / 50%	0.590	0.224	**2.64**×	2.4 MB（小于 L3）
250k / random / 50%	1.550	0.762	2.03×	6 MB（≈L3）
500k / random / 50%	3.162	1.472	2.15×	12 MB
1M / random / 50%	6.220	2.961	2.10×	24 MB
2M / random / 50%	12.683	5.985	2.12×	48 MB
10M / random / 50%	64.448	30.645	**2.10**×	240 MB
1M / random / 0%	7.140	2.640	**2.70**×	24 MB
1M / random / 100%	3.342	3.413	**0.98**×（退化）	24 MB
2M / blocks / 90%	7.168	6.888	**1.04**×（退化）	48 MB

逐样本交替测量（更严格）得到的比值：

实验	A 中位 (ms)	B 中位 (ms)	中位数之比	逐轮比区间	A 更慢轮数	符号检验 p
1M / 50%（20 轮 × 6）	11.466	6.772	**1.69**×	[1.30, 2.32]	20/20	1.9e-06
10M / 50%（8 轮 × 4）	106.070	61.980	**1.71**×	[1.47, 1.91]	8/8	7.8e-03
1M / 0%（10 轮 × 4）	10.410	5.167	**2.02**×	[1.44, 2.58]	10/10	2.0e-03
1M / 100%（10 轮 × 4）	6.939	7.703	**0.90**×	[0.71, 1.25]	2/10	n.s.
2M / blocks / 90%（12 轮 × 4）	15.241	13.791	**1.11**×	[0.84, 1.35]	11/12	6.4e-03

怎么读这两张表（重要）：

- 比值本身不是常数：同一份 1M/50% 数据，在**更空闲**的会话里测得 **2.11**×（A 中位 6.46 ms），在**有宿主负载**的会话里测得 **1.69**×（A 中位 11.47 ms）。**报告头条数字 2.52× 是"顺序 A/B + 每轮取 5 次最小值"两个偏乐观因素叠加的产物**，可复现的区间是 **1.7**×~**2.1**×，随机器状态移动。
- **10M 在空闲机上同样是 2.10**×（64.4 → 30.6 ms），不再像负载态那样掉到 1.5×。第二轮"10M 因带宽受限而摊薄到 1.84×"的解释**只在负载态成立**。
- **退化场景只有"命中率接近 100%"**：100% 命中时优化实现略慢约 2~10%（多写的代价没有分支收益来补偿），blocks/90% 从"无差异"变为"1.0~1.1× 的微弱优势"（第二轮把 blocks/90% 的 1.02× 解释为"分布让分支极易预测所以优化无从发挥"，该解释在空闲态下仍成立——只是基线仍然略慢）。

7.3 消融实验：把"无分支搬运"拆成可单独计时的两步

5 个变体（脚本 `bench/micro_variants.cpp`，核心循环 `bench/micro_core.cpp`）：① baseline 原样 → ② 仅阈值局部化 → ③ 谓词改按位与（写出仍在分支内）→ ④ ③ + 无条件写 + 掩码推进指针（即当前默认实现）→ ⑤ ④ 的等价写法。

中位数（ms），括号内为相对 ① 的比值：

数据集	① baseline	② +阈值局部化	③ 谓词按位与	④ +无条件写	⑤ 等价写法
1M / random / 50%	10.719 (1.00×)	11.747 (0.91×)	10.669 (**1.00**×)	**6.424 (1.67×)**	5.890 (1.82×)
2M / blocks / 90%	14.090 (1.00×)	16.683 (0.84×)	15.246 (0.92×)	14.212 (0.99×)	15.183 (0.93×)
1M / random / 0%	11.687 (1.00×)	10.890 (1.07×)	8.259 (1.42×)	**5.289 (2.21×)**	4.970 (2.35×)
1M / random / 100%	8.727 (1.00×)	8.394 (1.04×)	8.778 (0.99×)	9.209 (0.95×)	8.198 (1.06×)

归因拆分：

数据集	①→③（谓词改无分支）	③→④（写出改无分支）	①→④（合计）
1M / random / 50%	1.005×（+**0.5%**）	**1.661**×（+**66%**）	1.669×

数据集	①→③（谓词改无分支）	③→④（写出改无分支）	①→④（合计）
2M / blocks / 90%	0.924×（−8%）	1.073×（+7%）	0.991×
1M / random / 0%	1.415×（+42%）	1.562×（+56%）	2.210×
1M / random / 100%	0.994×（−1%）	0.953×（−5%）	0.948×

结论（本节是对第二轮的正式更正）：

1. **把谓词写成按位与，在随机命中分布下几乎不产生收益**（1M/50%：+0.5%，与噪声同量级）；把谓词写成 `&&` 短路链反而"免费"占了便宜——因为短路可以在第一个不满足的条件处直接跳出，省掉后面的加载与比较。
2. **收益几乎全部来自"消除写出分支"**：1.67× 里有 1.66× 来自这一步。机理是：baseline 的循环体中有一个**数据相关、方向频繁翻转**的分支控制着"要不要写输出"，它与命中序列的游程长度直接相关；改成"无条件写 + 把掩码加到写指针上"后，循环体内**不再有任何数据相关分支**，乱序执行窗口不再被分支解析打断。代价是多写了字节（1M/50% 多写约 12 MB）并多一条指令。
3. **阈值局部化（②）不仅无收益，多数情况是负收益**（6 组测量中 4 组更慢）。第二轮"约 26%"的说法应作废：`-O2` 本就能把 `Query` 的常量字段提出循环，手工局部化反而改变了寄存器分配，并抑制了短路带来的"少加载"优势。
4. ⑤ 与 ④ 数值等价但写法不同，实测在多数场景略优（1M/50%：1.82× vs 1.67×），差异在噪声范围内，**不作为结论**——保留它是为了说明"性能对写法细节敏感，单次对比不足以断言"。

7.4 为什么这个解释更可信：它与"命中可预测性"完全吻合

用 `bench/analyze_data_structure.py` 直接统计输入数据的结构：

数据集	时间戳通过	延迟通过	flags 通过	联合命中	命中游程平均长度	命中切换频率
1M / random / 50%	83.35%	83.33%	83.33%	50.01%	**2.00 条**	**50.0%**
2M / blocks / 90%	96.67%	96.66%	96.68%	90.00%	**5000 条**	0.02%
1M / random / 0%	66.67%	66.61%	66.71%	0%	常数	0%
1M / random / 100%	100%	100%	100%	100%	常数	0%

把"被消除的那个分支有多难猜"与实测加速比对起来：

情形	被消除的写出分支	谓词分支	实测 ①→④
random 50%	游程 2.0 条、每两条翻转 → **极难预测**	83% 偏置	**1.67**×
random 0%	方向恒定，但四个谓词**从未全部通过**（66.7% 偏置）	66.7% 偏置、难预测	**2.21**×
blocks 90%	游程 5000 条 → 近乎完美预测	96.7% 强偏置	0.99×
random 100%	方向恒定、谓词全通过 → **全部完美预测**	100%	0.95×

即：**只有当"写不写"这件事本身难以预测时，无分支搬运才有价值**。这解释了为什么收益集中在中等选择性（约 1%~90%）与随机分布上，而在高命中率、强块结构下退化为 1.0× 甚至略亏。

7.5 结论不变项与更正项

项目	第二轮结论	第三轮结论
输出正确性（逐字节、全字段、保序、不去重）	成立	**不变**（公开测试 7 项 + 回归 7 项 + 60 组差分 + 5 数据集指纹全过）
加速比	1M 2.52×、10M 1.66~1.84×	**可复现区间 1.7**×~**2.1**×；空闲态 1M/10M 均约 **2.10**×；随宿主负载移动

项目	第二轮结论	第三轮结论
加速来源	谓词分支错预测 + 阈值局部化	**写出分支的消除**（贡献 >95%）；谓词改写 ≈0；阈值局部化 ≈0 或负
退化场景	blocks/90% 1.02×、命中 100% 0.91×	**只有命中 100% 明确退化（0.90~0.98×）**；blocks/90% 在空闲态为 1.04~1.11×
10M 掉速	带宽受限导致降到 1.84×	**负载态才成立**；空闲态 10M 仍是 2.10×
`twopass` 在 2 核上劣于单遍	成立（基线为负载态 55.07 ms）	**成立且更强**（74.6 ms 最小值仍优于两遍全部配置）
候选二（SoA）、候选五（32 位快路径）	"实测否定"	**降级为"未验证"**（原始样本缺失），不采用的理由改为"未纳入实现范围"
输入来源（`--count` 现场生成 vs `--input` 文件）	"现场生成系统性更快，不可混比"	**更正**：两者数据内容相同、**速度无系统性差异**（逐样本交替：100k 0.98×、1M 1.02×，符号检验均不显著，见 5.4 与 `results/source-generated-vs-file.txt`）；仍统一用 `--input` 的理由是**可归档可核对**，而非更快

7.6 已知局限与下一步

- 本机仅 2 核、宿主负载不可控，绝对耗时在 6.46~26.65 ms 之间漂移；**所有加速比都必须连同"机器状态"一起引用**。理想的验证环境是独占的核心数与可关闭的频率调节（本次不具备）。
- 容器仍禁用 `perf`（`perf_event_paranoid=4`），错预测率等硬件计数器数据无法采集；本节对分支代价的归因是**"消融 + 数据游程结构"的间接证据**，不是指令级归因。
- 若要恢复候选二、候选五的确定结论，需要：(a) 实装 `--impl wide` 与 SoA 布局变体；(b) 在 `-O2` 下确认是否真的存在可用的向量化路径（GCC 13 在 `-O2` 不做 reduction 向量化，需用 `-ftree-vectorize` 或手写 intrinsics 验证）。这两项本次未做。

8. 复现入口（全部命令均已从解包后的提交包实测通过）

前置说明（三条，否则会踩坑）：

1. 提交包**不含数据集与二进制**（平台规范）：`data/`、`gold/`、`out/` 需由脚本创建；二进制需先编译。
2. 请解包到**持久目录**（不要用 `/tmp`——某些环境会清理临时目录，导致解包内容消失）。
3. 性能类命令要求**机器空闲**（`uptime` 看 load1 < 1.0）；`perf.sh` 会自检并在不满足时**拒绝运行**（退出码 3）。

```
# ① 解包与构建
tar -xzf HPC-3-submit.tar.gz -C <目标目录> && cd <目标目录>/HPC-3-submit
g++ -O2 -std=c++17 -pthread src/optimized.cpp  -I src -o bench/optimized
g++ -O2 -std=c++17 -pthread bench/baseline.cpp -I src -o bench/baseline
#   等价 CMake 路径：  cmake -S . -B build && cmake --build build    （产物在 build/）

# ② 正确性完整复检（约 35 秒；缺数据集时自动补建，产物 results/verify-quick.txt）
bash bench/verify.sh

# ③ 性能完整复现（约 10–15 分钟；六步，产物写 results/ab-live/，不覆盖已有证据）
PERF_ASSUME_YES=1 bash bench/perf.sh 4 20
#   想更快：  bash bench/perf.sh 3 6（约 5 分钟）

# ④ 单点最可信判据（自动落盘；分析只读本次测量）
bash bench/interleaved.sh data/n1m-r50.bin 20 6
python3 bench/analyze_interleaved.py results/interleaved-live.txt

# ⑤ 图表（make_charts 数值内嵌、可逐字节复现；charts_from_data 从原始样本现算，交叉校验）
```

```
python3 bench/make_charts.py
python3 bench/charts_from_data.py
```

通过判据:

步骤	判据
②	`Ran 7 tests ... OK` ×2、`总计 60 次比对，失败 0 次`、5 个数据集 `逐字节一致`
③	六个步骤标题齐全、末尾打印 `全部性能实验完成`、退出码 0
④	`[数据来源] … 1 段实验`、段名 `data/n1m-r50.bin`、中位之比 **1.7–2.2**×、符号检验 `显著(p<0.01)`
⑤	生成 `results/chart-*.svg` 与 `chart-*-data.svg`

已归档的完整流程记录：`results/perf-full-run-verification.txt`（第三轮验收性重跑）与本轮干净机器上的六步输出（5.4/5.5/7.4 节引用的数据同源）。

关于 `bench/run_all.sh`：它是最早的"一条龙"脚本（编译 + 生成数据 + 全部实验），现在仍可用，但**它生成的是第二轮/第四轮的证据文件**（`ab-*.txt`、`exp-threads-10m.txt`、`scale-sweep.txt`、图表）。若只关心正确性与主性能结论，用上面的 ②③ 即可；若要重放旧证据，再用 run_all.sh。